

Class6: R functions

Seunghoon Oh (PID: A19372132)

Table of contents

Background	1
A first function	1
A generate_dna() function	3
A generate_protein() function	5
Generate random protein sequences of length 6 to 13	6
Q5: BLASTp search against nr — are your peptides “unique in nature”?	6
Connecting your findings to immunology (MHC class II and T-cell activation)	8

Background

All functions in R have at least 3 things:

- a *name* (we pick that and use it to call the function),
- input *arguments* (one or more comma separated inputs that go inside the brackets when we call the function),
- the *body* (the lines of R code that do the work of the function)

A first function

Here we will create a function to add some numbers. Let's call it `add()`.

Input arguments can be either “required” or “optional”. The latter have fall-back default values that will be used if the user does not specify them.

```
add1 <- function(x, y=0) {  
  x+y  
}
```

Can we use our new function:

```
add1(10,100)
```

```
[1] 110
```

```
add1(10)
```

```
[1] 10
```

Q1b: For your second version, adapt your first function so it can take a single input vector or two inputs as before. For example, `add(4, 7)` and `add( c(4,7,10) )`. [1 pt]

For the second version,

```
add2 <- function (x, y=0) {  
  sum(x, y)  
}
```

```
add2(4, 7)
```

```
[1] 11
```

```
add2(c(4,7,10))
```

```
[1] 21
```

Q1c: Finally, on your own (outside of class) create a third version of your function that can add any number of inputs that the user provides. For example, `add(1, 2, 3, -4)` should return 2. Hint: explore the `...` (dots) argument or the base R function `sum()`. [2 pts]

```
add3 <- function (x1, x2=0, ...) {  
  sum(x1, x2, ...)  
}
```

```
add3(4, 7)
```

```
[1] 11
```

```
add3(c(4,7,10))
```

```
[1] 21
```

```
add3(1, 2, 3, -4)
```

```
[1] 2
```

We can explicitly set a **return** value output from a function (rather than the default last line) by using the **return()** function call.

A generate_dna() function

A useful function here is the “base R” **sample()** function:

```
sample(1:5, size = 4, replace=TRUE)
```

```
[1] 4 5 4 5
```

We can use this to make a random nucleotide sequence if we draw from “A”, “C”, “G” and “T”...

```
sample(x=c("A","C","G","T"), size=60, replace=TRUE)
```

```
[1] "G" "T" "C" "T" "G" "T" "T" "G" "C" "T" "T" "C" "T" "A" "T" "C" "C" "T" "T"  
[20] "G" "G" "G" "G" "C" "T" "G" "T" "T" "T" "G" "C" "T" "C" "G" "C" "C" "A" "A"  
[39] "A" "C" "T" "A" "A" "A" "A" "A" "A" "T" "T" "G" "T" "C" "T" "A" "C" "T" "G"  
[58] "T" "T" "T"
```

Q2a: Write a function **generate_dna()** that returns a random DNA sequence of a length specified by the user. Your first version should return a multi-element vector of single character nucleotides. For example **generate_dna(6)** might return “A”, “T”, “T”, “G”, “A”, “C”. [1 pt]

```
generate_dna_2a <- function(length_dna){  
  sample(x=c("A","C","G","T"), size=length_dna, replace=TRUE)  
}
```

```
generate_dna_2a(60)
```

```
[1] "A" "T" "A" "G" "A" "A" "A" "T" "C" "A" "C" "T" "T" "T" "A" "G" "G" "G" "C"
[20] "G" "C" "T" "C" "C" "T" "C" "T" "C" "C" "C" "A" "C" "A" "A" "A" "C" "G" "T"
[39] "C" "G" "G" "A" "T" "G" "T" "T" "C" "A" "A" "A" "C" "T" "T" "A" "G" "C" "A"
[58] "C" "T" "T"
```

Q2b: Your second version should optionally be able to return either a multi-element vector of single character nucleotides (as before) or a single character string (not a vector of single letters but a single vector of multiple letters). For example “AAGGCTG”. [1 pt]

Functions that could be useful here are `paste()`, `if()`, and `return()`.

```
generate_dna_2b <- function(length_dna, single_string=FALSE){
  result_vec <- sample(x=c("A","C","G","T"), size=length_dna, replace=TRUE)

  if (single_string){
    return(paste(result_vec, collapse=""))
  }
  else {
    return(result_vec)
  }
}
```

```
generate_dna_2b(60)
```

```
[1] "C" "T" "T" "C" "C" "T" "G" "C" "C" "C" "A" "G" "A" "A" "G" "A" "G" "A" "T"
[20] "G" "T" "T" "T" "T" "C" "A" "C" "G" "A" "C" "G" "G" "A" "A" "G" "C" "T" "C"
[39] "T" "T" "T" "G" "G" "C" "A" "C" "C" "A" "C" "A" "C" "T" "A" "G" "C" "G" "C"
[58] "C" "G" "G"
```

```
generate_dna_2b(60, single_string=TRUE)
```

```
[1] "GGTTCCTCCGTACTGTAGCGTCCGTCTTGAGACAATTTGCATTTAGCAACACCGATGTCGT"
```

Q2c: Finally, create a final version of your function that prints out a FASTA format sequence with an id line indicating the sequence length.

I created the version to automatically print out the FASTA format if the `single_string` argument is set as `TRUE`.

```
generate_dna_2c <- function(length_dna, single_string=FALSE, FASTA=TRUE){
  result_vec <- sample(x=c("A","C","G","T"), size=length_dna, replace=TRUE)

  if (single_string){
    result_fastaseq <- paste(result_vec, collapse="")
    cat(">len",length_dna,"\n", result_fastaseq, sep="")
  }
  else {
    return(result_vec)
  }
}
```

And the example result is:

```
generate_dna_2c(9, single_string = TRUE)
```

```
>len9
TTGGCGCTT
```

A generate_protein() function

Q3. Write a function `generate_protein()` that returns a random peptide/protein sequence of a length specified by the user. For example `generate_protein(6)` might return "WQRTAG".

The function first generates the `result_protein` vector by sampling among 20 amino acids with the given size argument (the length). `replace` argument in the `sample()` should set as "TRUE", since same amino acids may be present in this amino acid sequence for the protein. Then we add up all the vector components - the single characters - into a single sequence string.

```
generate_protein <- function(length_protein){
  result_protein <- sample(x=c("A", "R", "N", "D", "C", "E", "Q", "G", "H", "I", "L", "K", "T", "V", "W", "Y", "M", "P", "F", "S"), size=length_protein, replace=TRUE)
  return(paste(result_protein, collapse=""))
}
```

```
generate_protein(6)
```

```
[1] "LGDCKG"
```

Generate random protein sequences of length 6 to 13

Q4. Adapt and use your `generate_protein()` function to generate a series of random protein sequences ranging from 6 to 13 amino acids in length (one sequence per length). Take advantage of the base R function `for()` or `sapply()` so that you do not have to call `generate_protein()` eight times by hand. Be sure to format your results in FASTA format ready to paste or upload as a query to NCBI BLASTp.

Since we have our `generate_protein()` function already, this is relatively simple. You just have to apply the `for()` to iterate between different lengths.

```
for(n in 6:13) {  
  cat(">id.",n,"\n", generate_protein(n),"\n", sep="")  
}
```

```
>id.6  
WFAYGV  
>id.7  
MKVVSGD  
>id.8  
WTIMNHKD  
>id.9  
EEGATSFHM  
>id.10  
MESRIHEAID  
>id.11  
AQPNSPMRSKH  
>id.12  
QYFTKYMMGEM  
>id.13  
PVGFHSMRMNSQ
```

Q5: BLASTp search against nr — are your peptides “unique in nature”?

```
BLASTp_Result <- data.frame(  
  Length = 6:13,  
  Identity = c(100.00, 100.00, 87.50, 100.00, 88.89, 80.00, 100.00, 88.89),  
  Coverage = c(100, 100, 100, 89, 90, 91, 75, 69)  
)
```

```
BLASTp_Result$Unique <- ifelse(BLASTp_Result$Identity == 100.00 & BLASTp_Result$Coverage == 100, "Y", "N")
colnames(BLASTp_Result) <- c("Length (aa)", "Best hit % identity", "Best hit % coverage", "Unique")
BLASTp_Result
```

	Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
1	6	100.00	100	N
2	7	100.00	100	N
3	8	87.50	100	Y
4	9	100.00	89	Y
5	10	88.89	90	Y
6	11	80.00	91	Y
7	12	100.00	75	Y
8	13	88.89	69	Y

Now you can visualize this dataframe as a markdown table using `knitr` package and `kable()` function within the package.

```
library(knitr)
knitr::kable(BLASTp_Result)
```

Length (aa)	Best hit % identity	Best hit % coverage	Unique? (Y/N)
6	100.00	100	N
7	100.00	100	N
8	87.50	100	Y
9	100.00	89	Y
10	88.89	90	Y
11	80.00	91	Y
12	100.00	75	Y
13	88.89	69	Y

Q5a: At which sequence length do your randomly generated peptides start to look “unique in nature” (i.e. no 100% coverage + 100% identity hit)? [1 pt]

: Starting at length **8** from my random-generated peptide sequences.

Q5b: Speculate **why** very short random peptides are almost always found in nr, while longer ones typically are not. Your answer should refer both to the size of the sequence space (20^L for a peptide of length L) and to the size of the known protein universe. [1 pt]

: The size of the sequence space **increases exponentially** as the peptide length L increases, while the size of the known protein universe is finite: in other words, when we are using BLASTp to compare our generated peptide sequence to the known *Non-redundant protein sequences (nr)* database, at some point (length) the sequence space size exceeds the size of the *nr* database. This makes the probability of our peptide being **not unique** to exponentially decrease, and most of the peptides above certain length will be ~100% unique from the BLASTp results. Thus very short random peptides, having a smaller (or similar) sequence space compared to *nr* database, are always found in *nr*, while longer ones are mostly found unique, and the threshold of this “**space size order reversal**” seems to lie around the length (L) 8-9 from my results.

Connecting your findings to immunology (MHC class II and T-cell activation)

Q6: In 3–6 sentences total and using your Q5 data and the reasoning from Q5b, what do you think this minimum length is and why might it be a bad design choice for the immune system to present very short peptides? [3 pt]

: From the answer for Q5b, I estimated the threshold length for “sequence space size order reversal” to be around 8-9 from my data. Since MHC-presented peptides should be distinguished by the T-cell receptor if it's self or non-self, the minimal length should be set to make the peptide unique from each pool of “self” and “non-self” peptides. If the displayed peptide length is too short, the receptors might misconceive the self peptide as non-self, or vice versa - since the peptides are not unique. Since some students got the unique peptides starting from length 9, I would like to be conservative for my estimate: the **minimal length may be 9**.